

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 24/09/25	

Instrucciones

Lea atentamente las instrucciones siguientes antes de continuar:

- Para este examen **solo se permite el uso de material de escritura**, como bolígrafos, lápices, gomas o cinta correctora. No está permitido usar dispositivos electrónicos de ningún tipo, apuntes ni ningún otro material distinto del indicado. **No se corregirá ninguna parte del examen escrita en lápiz.**
- Este examen se compone de 5 **hojas**, que se entregan grapadas y que **no deberá desgrapar.**
- El **anverso** de cada hoja (es decir, la página que contiene texto impreso y un encabezado, la que está leyendo ahora mismo) contiene tanto los **enunciados** de los problemas como **espacio para responder. Solo se corregirán estos espacios.** Antes de continuar leyendo, por favor, compruebe que su examen tiene 5 hojas y escriba su nombre completo y DNI en todas ellas.
- El **reverso** de cada hoja (es decir, la página que está completamente en blanco) es un espacio en blanco que podrá utilizar para lo que considere oportuno, como **cálculos o notas en sucio. Nada de lo que aparezca en estas páginas será corregido.** Únicamente si rellena por completo todos los reversos de su examen se le proporcionará una hoja de sucio adicional; levante la mano para que un profesor se la entregue.
- Al finalizar el examen deberá **tomar una fotografía de todos los anversos** (incluyendo esta página que está leyendo ahora mismo). Sabrá que una página tiene que ser fotografiada porque contiene un encabezado como el de esta.
- La correcta interpretación del enunciado se considera parte de las competencias a evaluar en el examen.
- Durante el examen aparecerán fragmentos de código, si en alguna parte de ellos aparece el texto "...", eso indicará que dicho fragmento en concreto no se proporciona porque no es relevante para la resolución del ejercicio.
- Cada ejercicio incluye todas las clases y métodos necesarios para resolverlo que no pertenezcan a las librerías estándar de Java.
- Cada ejercicio muestra su contribución a la puntuación máxima del examen, que es de **10 puntos.**

Ya puede comenzar el examen. ¡Buena suerte!

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 24/09/25	

Ejercicio 1: 5 puntos

En la práctica 1 se trabaja con datos de electroencefalogramas (EEG). Cada sesión de EEG se representa mediante un objeto de la clase `EEGModel`, que contiene una `List` de objetos de la clase `Measurement`. Cada `Measurement` representa una muestra, que contiene a su vez un `float[]` para almacenar los valores de los canales. Esto se puede ver en el siguiente código:

```
public class EEGModel {
    private List<Measurement> measurements;

    public EEGModel(List<Measurement> measurements) { ... }
    public List<Measurement> getMeasurements() { return measurements }
}

public class Measurement {
    private float[] channels;

    public Measurement(float[] channels) { ... }
    public float[] getChannels() { return channels; }
}
```

La práctica también introduce el concepto de filtros para procesar datos. Un tipo de filtro útil podría ser uno que elimine las muestras que no alcanzan un determinado `umbral de activación en ciertos canales`, conservando solo aquellas con señales fuertes. Por ejemplo, en el siguiente listado de 3 muestras, cada una con 4 canales:

```
{83.0, 214.0, 57.0, 70.0}
{2.0, 131.0, 60.0, 73.0}
{16.0, 47.0, 55.0, 2.0}
```

Si los canales especificados son el 0 y el 3, y el umbral un 10.0, tras filtrar quedaría únicamente la muestra que supera el umbral en esos canales:

```
{83.0, 214.0, 57.0, 70.0}
```

Deberá implementar el método `applyFilter` de este filtro, que recibe como `argumento un EEGModel` sin filtrar y debe `devolver un EEGModel`, nuevo y filtrado. El método pertenece a una clase `ThresholdFilter` cuyo constructor ya inicializa `dos atributos de la clase: threshold, un float` con el umbral del filtro; y `channels`, un `int[]` con los índices de los canales que deben superar el umbral. El siguiente código marca con un `TODO` lo que debe hacer: `Recorre cada Measurement de la lista. Si los canales correspondientes de la medida superan el umbral, se añade a la nueva lista de medidasFiltradas`

```
public class FilterThreshold implements Filter {
    private float threshold; // Umbral a superar
    private int[] channels; // Canales en los que se aplica el umbral

    public FilterThreshold(float threshold, int[] channels) { ... }
    public EEGModel applyFilter(EEGModel eeg) { // TODO: Implemente este método
    }
}
```

```
public EEGModel applyFilter(EEGModel eeg){
    List<Measurement> medidasOriginales = eeg.getmeasurements();
    List<Measurement> medidasFiltradas = new ArrayList <>();
    for (Measurement medida : medidasOriginales){ //recorrer las medidas de la lista
        float[] canalesOriginales = medida.getchannels(); // obtener los canales de cada medida
        for (int i: canalesOriginales){ //recorrer cada canal de la medida. Comprobar si el canal en la posicion dada por int[]
            channels supera el umbral. Al comparar indices = usar indices en for
                for (int j: canales){
                    if (i = channels[j] && canalesOriginales[j] > threshold) medidasFiltradas.add (medida);
                }
            }
        }
    }
    EEGModel eegFiltrado = new EEGModel(medidasFiltradas);
    return eegFiltrado;
}
```

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 24/09/25	

Escriba su respuesta a continuación:

```
public EEGModel applyFilter(EEGModel eeg) {  
    // Lista de muestras originales  
    List<Measurement> ogMeasurements = eeg.getMeasurementsList();  
    // Lista de muestra filtradas, inicialmente vacía  
    List<Measurement> filteredMeasurements = new ArrayList<Measurement>();  
    // Recorremos las muestras originales  
    for (Measurement measurement : ogMeasurements) {  
        // Indica si vamos a eliminar la muestra  
        boolean filtrado = false;   
        // Extraemos los canales de la muestra  
        float[] ogChannels = measurement.getChannels();  
        // Recorremos los canales de la muestra  
        for (int i = 0; i < ogChannels.length; i++) {  
            // Recorremos los canales a comprobar  
            for (int j = 0; j < channels.length; j++) {  
                // Si es uno de los canales a comprobar y no alcanza  
                // el umbral, lo marcamos para filtrado  
                if (i == channels[j] && ogChannels[i] < threshold) {  
                    filtrado = true;  
                }  
            }  
        }  
        // Si no ha sido marcada para filtrar, añadimos la muestra a la nueva lista  
        if (!filtrado)  
            filteredMeasurements.add(measurement);  
    }  
    // Crea y devuelve el nuevo modelo  
    EEGModel filteredModel = new EEGModel(filteredMeasurements);  
    return filteredModel;  
}
```

TIENE QUE COMPROBARSE EN TODOS LOS CANALES
→ en principio marcar como si no se elimina (asumir que es correcto)
→ si no cumple UN canal
↓
marcar como filtrar

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 24/09/25	

Ejercicio 2: 5 puntos

Los datos de la práctica 1 se leen y se escriben en archivos de texto en un formato específico, en el que cada línea representa una medida en un instante de tiempo. Cada línea contiene varios valores separados por comas, siendo el primer valor el índice de la muestra. Este índice empieza en 0 en la primera línea y se va incrementando en las líneas siguientes en módulo 256 (es decir, que aumenta hasta 255 y luego vuelve a 0). Este valor inicial está seguido de otra serie de valores separados por comas, en el que cada uno representa un canal. Un archivo puede tener tantos canales como se quiera. El siguiente fragmento de archivo es un ejemplo de este formato, en este caso con 4 canales. Los “...” indican partes del fichero que se omiten:

```
0, 214.2, -8.5, 57.9, 70.7
1, 131.0, 2.1, 60.3, 73.3
2, 79.4, 16.8, 83.3, 51.0
...
255, 2.0, 131.6, 60.0, 73.0
0, 16.0, 47.1, 55.0, 2.3
1, 24.7, 36.0, 456.0, 245.5
...
```

Deberá implementar un método estático de la clase `EEGModel` llamado `checkConsistency`, que recibe como parámetro un `String` que representa el contenido de unos de estos archivos de datos. Debe devolver `true` si todos los índices del archivo son consecutivos y `false` en caso contrario. Se puede asumir que el primer índice de muestra siempre es 0. Por ejemplo, para el siguiente `String` de entrada, el método debería devolver `false` (ya que después del índice 1 debería venir 2 y, en lugar de eso, viene el 3):

```
0, 214.2, -8.5, 57.9, 70.7
1, 131.0, 2.1, 60.3, 73.3
3, 79.4, 16.8, 83.3, 51.0
```

También deberá devolver `false` si uno de los índices no es un número entero. Por ejemplo, para el siguiente `String` de entrada, el método debería devolver `false`, pues uno de los índices no es un número entero, sino el texto “ERROR”:

```
0, 214.2, -8.5, 57.9, 70.7
ERROR, 131.0, 2.1, 60.3, 73.3
3, 79.4, 16.8, 83.3, 51.0
```

El siguiente código marca con un `TODO` lo que debe hacer:

```
public static boolean checkConsistency(String contenidoArchivo) {
    // TODO: Implemente este método
}
```

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 24/09/25	

Si lo necesita, puede utilizar los siguientes métodos de la clase String:

```
String[] split(String d); // Divide un String en un String[] usando 'd' como separador.  
// Ej: Con d = ".", "a.b.c" pasaría a ser el array {"a", "b", "c"}  
String trim(); // Devuelve un String igual al original pero eliminando los espacios en  
// blanco al principio y al final. Ej: " a " pasa a ser "a"
```

Recuerde que el carácter de salto de línea es `"\n"`.

También puede utilizar el siguiente método **estático** de la clase Integer:

```
static int parseInt(String s); // Si 's' contiene un número entero, devuelve un int con ese  
// valor. Ej: Si s = "5", devolverá un int = 5  
// Lanzará NumberFormatException si 's' no contiene un entero
```

Escriba su respuesta a continuación:

↳ manejar excepción

```
public static boolean checkConsistency(String contenidoArchivo) {  
    // Dividimos el contenido en líneas.  
    String[] lineas = contenidoArchivo.split("\n");  
    int indiceEsperado = 0;  
    for (String linea : lineas) {  
        // Dividimos la línea por las comas para obtener los valores.  
        String[] valores = linea.split(",");  
        try {  
            // Parseamos el primer valor (índice) a entero.  
            int indiceActual = Integer.parseInt(valores[0].trim());  
            // Comparamos el índice actual con el que esperamos.  
            if (indiceActual != indiceEsperado) {  
                return false; // No son consecutivos.  
            }  
            // Incrementamos el contador para la siguiente línea.  
            indiceEsperado = (indiceEsperado + 1) % 256;  
        } catch (NumberFormatException e) {  
            // Si el primer valor no es un número, el formato es incorrecto.  
            return false;  
        }  
    }  
    // Si hemos recorrido todas las líneas y los índices eran correctos,  
    // la consistencia es correcta.  
    return true;  
}
```